

CI CD Delivery Handbook

A start to finish workflow for the CERA Medical development team

A beginner-friendly operating guide for building, reviewing, testing, staging, releasing, deploying and recovering the CERA platform with GitHub and Docker Compose.

Audience	Applies to	Version
Three-person development team	Next.js, Vendure, Payload CMS, PostgreSQL, Redis or Valkey, Authentik, Cloudflare R2, GlitchTip, Zoho, Resend, Cloudflare, Caddy, Docker Compose and Hetzner	Checked 17 September 2026

Prepared by Stack & Scale

Build today. Scale tomorrow.

How to use this handbook

The shortest useful summary is: work in a small branch, open a pull request, let automated checks run, get another person to review, merge to develop, test on staging, then release to main and production.

The CERA release train

1 Change	2 Review	3 Rehearse	4 Release
feature/* Developer laptop	Pull request CI and review	develop Staging server	main Production server

Remember A branch is a safe working copy of the repository. An environment is a place where code runs. They are related, but not the same: a branch decides which code is considered; an environment decides where it runs.

CERA defaults

Decision	CERA default
Primary branches	main is production; develop is shared integration and staging.
Feature branches	Short-lived feature/*, fix/*, chore/* and docs/* branches created from develop.
Merge path	feature -> PR -> develop -> release/* -> main. hotfix/* goes to main and then back to develop.
Deployment	develop deploys to staging; main deploys to production after protected review and environment approval.
Runtime	One versioned Docker image moves from CI to Hetzner; Caddy is public proxy; databases and files persist.

The stack in one picture

```

Internet users
|
Cloudflare DNS, CDN, WAF, rate limits
|
Caddy on Hetzner: HTTPS and reverse proxy
|
Next.js storefront ---- Vendure commerce API ---- PostgreSQL
|               |               |
Payload CMS ----- Authentik             Redis / Valkey
|
Cloudflare R2 files | Resend email | Zoho staff mail | GlitchTip errors
  
```

Who creates the repository and shared foundation

The technical lead or designated repository owner creates the GitHub organization, repository and initial branches. Other developers join after the foundation is pushed, so everyone starts from the same point.

Account ownership

Account	Owner	Team access
GitHub organization and repository	CERA owner or nominated technical lead	Named developers; no shared login.
Cloudflare, Hetzner, R2 and DNS	CERA owner	Named users or teams with least privilege and MFA.
Zoho and Resend	CERA operations owner	Operations and technical administrators; API keys only where needed.
Authentik and GlitchTip	Technical lead	Named admins, MFA and emergency access documented.
Password manager	CERA owner	Shared vault for secrets; never GitHub Issues or chat.

1. Create private repository `cera-platform` unless CERA approves another name.
2. Add README, CONTRIBUTING, SECURITY, CODEOWNERS, `.gitignore`, `.env.example`, issue templates and CI.
3. Add three developers, technical lead and CERA owner with minimum GitHub roles.
4. Turn on MFA for GitHub, Cloudflare, Hetzner, email, password manager and monitoring.
5. Agree protected branches, staging hostname, production hostname, release owner and emergency hotfix owner.

Ownership rule CERA must retain ownership of GitHub and infrastructure accounts. A vendor-owned personal account must never be the only recovery route.

Repository starter files

```

README.md
CONTRIBUTING.md
SECURITY.md
CODEOWNERS
.gitignore
.env.example
docker-compose.yml
docker-compose.staging.yml
docker-compose.production.yml
apps/storefront/ # Next.js
apps/commerce/  # Vendure server and worker
apps/cms/        # Payload CMS
packages/contracts/ # shared types and schemas
packages/ui/      # shared UI
infrastructure/caddy/ # Caddyfile
infrastructure/scripts/ # backup, health check, deploy helpers
docs/             # decisions, runbooks and contracts
.github/workflows/

```

Day 1 bootstrap and local setup

The first push is a small working skeleton. It does not need every feature; it needs shared names, commands and files that let three people work without guessing.

Exact commands for repository owner

```
mkdir cera-platform
cd cera-platform
git init -b main
git config user.name "Your Name"
git config user.email "you@example.com"
git add .
git commit -m "chore: initialize CERA platform"
git remote add origin git@github.com:CERA-ORG/cera-platform.git
git push -u origin main
git switch -c develop
git push -u origin develop
```

Exact commands for each developer

```
git clone git@github.com:CERA-ORG/cera-platform.git
cd cera-platform
git switch develop
git pull --ff-only origin develop
corepack enable
pnpm install
cp .env.example .env.local
docker compose up -d postgres redis
pnpm dev
```

Area	Primary owner	First shared output
Infrastructure	Developer A or technical lead	Compose files, Caddy, health checks and CI skeleton.
Storefront	Developer B	Next.js shell, UI tokens and mock-data adapters.
CMS and content	Developer C	Payload collections, roles, seed data and content contract.
Commerce contract	A with B	Vendure product/cart/order shape and errors.
Integration decisions	All three	One short decision record in docs/ before cross-cutting change.

Feature branches commits pull requests and reviews

Branches keep incomplete work away from shared branches. Pull requests are the team safety gate.

Normal feature loop

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/venture-foundation
git status
git add apps/commerce packages/contracts
git commit -m "feat(commerce): add initial Vendure module"
git push -u origin feature/venture-foundation
```

Command	Easy meaning
git switch develop	Move working folder to shared development branch.
git pull --ff-only origin develop	Download newest develop without surprise merge commit.
git switch -c feature/name	Create branch from current commit and move onto it.
git status	Show changed files and current branch.
git add path	Choose changes for next commit.
git push -u origin branch	Send branch to GitHub for review.

Naming and commits

- Good branches: feature/research-service-page, fix/checkout-tax-rounding, chore/upgrade-payload, docs/local-setup.
- Avoid feature/all-week-work. Keep one branch focused and short-lived.
- Use commits such as feat(storefront): add research service card, fix(checkout): reject expired cart token, chore(ci): cache pnpm store.
- Open a draft PR early when you need feedback. Delete feature branch after merging.

Pull request checklist

PR field	What author writes
Title and why	Short outcome plus customer or operational reason.
What changed	Routes, contracts, migrations, UI, configuration and risks.
How tested	Exact commands and local or staging URL.
Data and migration	PostgreSQL, Payload, Vendure or seed-data impact.
Screenshots	Before/after for UI; redact patient, customer and secret data.
Rollout notes	Feature flag, deployment order, backfill, rollback or follow-up issue.

- At least one other developer reviews every PR; author does not self-approve.
- Review behavior first, then security, data shape, errors, tests and documentation.
- Use CODEOWNERS for auth, database, deployment and shared-contract paths.
- CERA default: squash merge feature -> develop; release owner merges release -> main; hotfix is small and auditable.

Syncing develop and resolving merge conflicts

When develop moves forward, your feature branch does not automatically receive those changes. Syncing means bringing newest shared code into your branch before final review.

Exact sync commands

```
git switch develop
git pull --ff-only origin develop
git switch feature/vendure-commerce
git merge develop
git push origin feature/vendure-commerce
```

In plain language: go to shared branch, download newest version, return to work, bring shared changes into your branch, publish it and let CI test combined result.

Choice	What happens	CERA guidance
Merge develop into feature	Git creates a merge commit joining both lines.	Default for first 10 days and shared branches.
Rebase feature onto develop	Git rewrites feature commits after newest develop.	Optional for experienced developer on private branch; never force-push main/develop.
GitHub Update branch	GitHub can update PR branch if no conflicts.	Use documented option and re-run CI.

When Git reports a conflict

```
git merge develop
git status
# Open each conflicted file; choose correct final content.
# Remove: <<<<<<, =====, >>>>>>
git add path/to/resolved-file
git commit -m "chore: merge latest develop into feature branch"
pnpm lint
pnpm test
git push origin feature/vendure-commerce

# If you are lost:
git merge --abort
```

Team habit Ask the file owner if conflict changes a contract, migration, authentication rule or deployment setting. Small frequent syncs are easier than one giant sync after six days.

Parallel work with mock data and data contracts

A frontend does not wait for a finished CMS or commerce backend if both sides agree on interface first. The contract promises field names, types, optional values, errors and examples.

Example CERA contract

```
type ResearchService = {
  id: string;
  slug: string;
  title: string;
  summary: string;
  heroImageUrl?: string;
  capabilityBullets: string[];
  ctaLabel: string;
  updatedAt: string; // ISO 8601
};
```

Developer C builds Payload to return this shape. Developer B builds the Next.js page using mock data of the same shape. When API is ready, B replaces a mock adapter with a real adapter instead of rewriting the page.

Contract type	Location	Owner
Shared TypeScript types and Zod schemas	packages/contracts/	A coordinates; B and C contribute.
Human-readable examples	docs/contracts/research-service.md	Developer changing contract updates it.
API response tests	apps/storefront and apps/cms tests	Provider and consumer test each side.
Breaking change decision	docs/decisions/ADR-*.md	All three approve before merge.

```
export const mockResearchServices: ResearchService[] = [{
  id: "mock-preclinical-1",
  slug: "preclinical-studies",
  title: "Preclinical Studies",
  summary: "Evidence-led support for early biomedical research.",
  capabilityBullets: ["Study planning", "Biochemical testing"],
  ctaLabel: "Request a consultation",
  updatedAt: "2026-09-01T00:00:00.000Z",
}];
```

Safety rule Mock data contains no patient, customer or confidential research information. Never copy real records into developer laptops or PR screenshots.

Local staging and production environments

These are separate places where same application runs with different data, domains, credentials and risk. Promote same built image while changing configuration and persistent data.

Environment	Who sees it	Branch or trigger	Data and secrets
Local	One developer	feature/* on laptop	Disposable seed data; .env.local untracked and developer-specific.
Staging	Team, QA and approved reviewers	develop push or manual dispatch	Non-production credentials and sanitized seed data; protected environment.
Production	Real customers and CERA staff	main push after release approval	Real data and credentials; reviewers and one deploy at a time.

Deployment map

GitHub event	Build	Destination	Approval
PR to develop	Lint, types, tests, builds, dependency/secret checks	No deployment; preview only if approved	PR review and checks
Push to develop	Image tagged with commit SHA	Staging on Hetzner	Staging environment; no prod secrets
PR release/* -> main	Full CI and release checklist	No production deploy	Release owner plus CERA owner
Push to main	Exact commit image	ceramed.org production on Hetzner	Production reviewer; concurrency lock

.env handling

File or place	Commit to Git	Use
.env.example	Yes	Names, safe placeholders and comments; never credentials.
.env.local	No	Local values on one laptop; listed in .gitignore.
GitHub environment secrets	No	Staging/prod credentials only for matching deployment job.
/opt/cera/secrets	No	Runtime secrets on Hetzner with restricted permissions.
Docker Compose secrets	No	Use mounted files for high-value secrets when image supports them.

Important NEXT_PUBLIC_ variables are exposed to browser code and may be inlined at build. Treat them as public configuration, never secrets. Server-only values stay unprefixed and are read at runtime.

GitHub rulesets branch protections and security

Use GitHub Rulesets to turn team agreements into enforcement. UI labels can change; policy names below are what the team should configure.

Ruleset for main

Rule	Set it to	Why
Target branches	main	Protect production source of truth.
Enforcement	Active	Disabled rule is documentation only.
Require PR before merge	On	No direct production changes.
Required approvals	1 minimum; dismiss stale approvals	Second person confirms latest code.
Required status checks	lint, typecheck, test, build	Merge passes release checks.
Require branch up to date	On if CI is fast enough	Tests final combined state.
Block force pushes and deletion	On	Prevents history rewriting and loss.
CODEOWNERS review	On for deploy, auth, database and workflows	Routes sensitive changes.
Bypass list	CERA owner and emergency technical lead only	Use rarely; record and back-merge hotfix.

Ruleset for develop

- Require PR, one approval and same CI checks.
- Block force pushes and branch deletion so staging history is dependable.
- Require up-to-date branch when CI is fast enough; otherwise daily sync is mandatory.

```
/apps/commerce/ @cera/commerce-owner
/apps/cms/ @cera/cms-owner
/apps/storefront/ @cera/storefront-owner
/.github/workflows/ @cera/technical-lead
/infrastructure/ @cera/technical-lead @cera/cera-owner
/packages/contracts/ @cera/commerce-owner @cera/storefront-owner
```

Secrets policy

- Never commit secrets, paste them in chat or include them in screenshots.
- Use least-privilege credentials, separate staging and production values, and rotate on suspicion of exposure.
- PR workflows never receive production secrets.
- Start GitHub Actions permissions at contents: read. Pin production third-party actions to full commit SHA after reviewing source.
- Prefer GitHub-hosted runners for untrusted code; self-hosted runners are persistent and not isolated.

GitHub Actions continuous integration and continuous delivery

Continuous integration checks every proposed change. Continuous delivery promotes an immutable Docker image from develop to staging and from main to production.

Minimum CI checks

- Frozen dependency install.
- Formatting and lint.
- TypeScript check across apps/packages.
- Unit and integration tests with disposable PostgreSQL/Redis where needed.
- Next.js, Vendure and Payload build without production credentials.
- Dependency review and secret scanning; add end-to-end smoke tests after basic workflow is stable.

Beginner-friendly CI workflow

```
name: CI
on:
  pull_request:
    branches: [develop, main]
  push:
    branches: [develop, main]
permissions:
  contents: read
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with:
          version: 10
      - uses: actions/setup-node@v4
        with:
          node-version-file: .nvmrc
          cache: pnpm
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm typecheck
      - run: pnpm test -- --runInBand
      - run: pnpm build
```

The readable sample uses version tags. Before production, pin third-party actions to full commit SHAs, review source and keep approved update records. GitHub secure-use guidance treats SHA pinning and least privilege as stronger settings.

Hetzner deployment design

Part	CERA implementation
Build	Actions builds Next.js, Vendure and Payload images tagged with commit SHA.
Registry	GitHub Container Registry or approved private registry; never use latest alone.
Transport	GitHub-hosted runner connects to deploy-only SSH account on Hetzner. Account is not root.
Runtime	Docker Compose pulls exact tag, runs migrations in order, restarts services with health checks.
Edge	Caddy terminates HTTPS and proxies internal services; Cloudflare fronts DNS and edge protection.

Part	CERA implementation
Observability	GlitchTip, health endpoint, logs and uptime monitor checked after deployment.

```
# On the staging host, called by reviewed deploy script
cd /opt/cera/staging
export IMAGE_TAG=sha-abc123
docker compose pull
docker compose run --rm commerce pnpm migration:run
docker compose up -d
./scripts/smoke-check.sh
```

Production job references GitHub environment named production. Its protected SSH key, known-hosts entry and registry token are unavailable until deployment protection rules pass. Use a concurrency group so two production deploys cannot overlap.

CERA stack environment and integration checklist

Make difference between code configuration and secret values obvious. Each service has a small documented set of settings without exposing credentials.

Service	Local	Staging and production rule
Next.js	localhost:3000; mock adapters allowed	Behind Caddy; server-only secrets runtime; NEXT_PUBLIC_ is public.
Vendure	Local API plus seeded catalog	Server and worker separate when needed; persistent PostgreSQL; reviewed migrations.
Payload CMS	Local admin with seed content	Strong PAYLOAD_SECRET; access control and secure cookies behind HTTPS; Postgres adapter.
PostgreSQL	Docker volume with disposable seed	Separate database per environment; backups, restore test and migration log.
Redis or Valkey	Docker service	Cache, queues and sessions only; does not replace PostgreSQL durability.
Authentik	Local provider or mocked auth	Separate config/credentials per environment; MFA for admins; never copy prod users.
Cloudflare R2	Filesystem or test bucket	Private bucket; server credentials; short-lived presigned URLs; validate type/size.
GlitchTip	Console or local instance	Distinct DSNs; no sensitive payloads; alerts to on-call owner.
Zoho and Resend	Fake mailboxes/test domain	CERA owns DNS/mailboxes; verified domain, separate keys, webhooks and safe logging.
Cloudflare/Caddy/Hetzner	Local Caddy or localhost	Firewall, persistent Caddy data, SSH keys, backups, monitor and deploy account.

Safe .env.example

```

NODE_ENV=development
APP_URL=http://localhost:3000
DATABASE_URL=postgresql://cera:cera@localhost:5432/cera_local
REDIS_URL=redis://localhost:6379
PAYLOAD_SECRET=replace-with-a-local-only-value
AUTHENTIK_ISSUER_URL=http://localhost:9000/application/o/cera/
R2_ENDPOINT=https://<account-id>.r2.cloudflarestorage.com
R2_BUCKET=cera-local
RESEND_API_KEY=re_local_only
GLITCHTIP_DSN=

# Never commit real values.
# NEXT_PUBLIC_* values are visible in browser code.
```

Issues releases tags hotfixes and rollback

A release is a named testable point in history. It makes it possible to answer what changed, where it is deployed and how to recover.

Issue and task workflow

State	Meaning	Owner action
Backlog	Idea or requirement exists.	Write acceptance criteria and dependencies.
Ready	Small enough to build; questions answered.	Assign owner and link issue to branch.
In progress	Developer working.	Keep branch short; update if scope changes.
In review	PR open and CI running.	Request review; add test notes and screenshots.
Staging	Merged into develop and deployed.	Run acceptance checks and record defects.
Done	Accepted and released or deferred.	Close after target environment confirmed.

Release branch and tag

```
git switch develop
git pull --ff-only origin develop
git switch -c release/2026.09.0
git push -u origin release/2026.09.0

# Release blockers only, then PR release/* -> main.
git switch main
git pull --ff-only origin main
git tag -a v2026.09.0 -m "CERA release 2026.09.0"
git push origin v2026.09.0

# Back-merge release-only fixes to develop.
git switch develop
git pull --ff-only origin develop
git merge --no-ff release/2026.09.0
git push origin develop
```

Hotfix

```
git switch main
git pull --ff-only origin main
git switch -c hotfix/checkout-error-message
git add .
git commit -m "fix(checkout): handle payment timeout"
git push -u origin hotfix/checkout-error-message

# Emergency-reviewed PR to main, deploy, smoke test.
# Then merge same hotfix back into develop.
```

Rollback

```
docker compose images
# Set known-good image tag in environment file.
export IMAGE_TAG=sha-known-good
docker compose pull
docker compose up -d
./scripts/smoke-check.sh
docker compose logs --since 30m > /tmp/cera-rollback-logs.txt
```

Recovery rule Do not edit server files to fix production. If emergency bypass is necessary, record approver, change, commit/tag, smoke test and follow-up PR. Image rollback may not be safe after incompatible database migration; prefer expand-and-contract migrations.

Day 1 to Day 10 delivery plan

The first ten days establish the delivery habit. Merge small slices daily instead of holding three large branches until the end.

Role allocation

Developer	Primary ownership	Secondary review
A Infrastructure and commerce	Docker Compose, Hetzner, Caddy, CI/CD, PostgreSQL, Redis/Valkey, Vendure and releases.	Migrations, workflows, security and deployment.
B Storefront and experience	Next.js routes, UI, accessibility, product/research journeys, mocks and smoke tests.	CMS contracts, API errors and visuals.
C CMS identity and integrations	Payload, Authentik, roles, seed content, R2, GlitchTip, Zoho and Resend.	Auth, permissions, secrets, content and email.

Ten-day sequence

Day	Shared goal	Expected output
1	Repository/rules	Repo, main/develop, README, .env.example, skeleton, issue template, CI.
2	Local foundations	Compose PostgreSQL/Redis, Next.js shell, Vendure/Payload starters, health endpoints.
3	Contracts/identity	Product/research contracts, Authentik local setup, Payload roles and seed content.
4	Commerce slice	Vendure product read, catalog seed, product card, integration PR.
5	Research slice	Payload collection, service page, mock-to-real adapter PR.
6	Files/email	R2 private upload/download, Resend test email, GlitchTip errors.
7	Staging path	Hetzner staging Compose, Caddy, Cloudflare DNS, staging secrets, develop deploy.
8	Rehearsal	Login, CMS, product, checkout test mode, research lead and email smoke tests.
9	Release hardening	Production Compose, backups, monitoring, ruleset, approval and rollback rehearsal.
10	First release	Release branch, tag, production deploy, smoke test, handover and follow-up issues.

Daily rhythm

- Start: name issue, branch, contract dependency and expected PR time.
- During: commit small units; ask before shared-contract change; sync develop daily.
- Review: open draft PR early for cross-cutting work; ask one focused reviewer.
- End: merge only green reviewable work; update issue with done, blocked or next.

The practical answer If UI depends on CMS, UI developer continues with mock data while CMS developer builds real provider. The dependency is managed by contract, not by waiting for backend detail.

Checklists sources and maintenance

Keep this section open during first week. The commands repeat because repetition turns a workflow into habit.

Before starting feature

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/short-name
pnpm install --frozen-lockfile
docker compose up -d postgres redis
pnpm dev
```

Before opening PR

```
pnpm lint
pnpm typecheck
pnpm test
pnpm build
git status
git add path/to/your/files
git commit -m "feat(area): describe the change"
git push -u origin feature/short-name
```

Before merge and after deploy

- PR targets correct base branch and links issue.
- CI green and approval applies to latest commit.
- No secret, patient/customer data, build output or local .env included.
- Migration, permission and rollback notes are present.
- After deployment, check homepage, login, CMS read, product/cart path, research form, approved email, logs and GlitchTip.

Official sources checked 17 September 2026

- **GitHub rulesets:** <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets>
- **GitHub deployments/environments:** <https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments>
- **GitHub Actions secure use:** <https://docs.github.com/en/actions/reference/security/secure-use>
- **Next.js self-hosting:** <https://nextjs.org/docs/app/guides/self-hosting>
- **Payload production deployment:** <https://payloadcms.com/docs/production/deployment>
- **Docker Compose production and secrets:** <https://docs.docker.com/compose/how-tos/use-secrets/>
- **Caddy automatic HTTPS:** <https://caddyserver.com/docs/automatic-https>
- **Cloudflare R2 presigned URLs:** <https://developers.cloudflare.com/r2/api/s3/presigned-urls/>

Maintenance cadence

When	Review
Every PR	Workflow changes, secrets, migrations, auth and contracts have required owner review.
Monthly	Action pins, dependency alerts, base images, backups, monitors and access list.
Quarterly	Restore database backup, rehearse rollback, review secrets, rotate deploy credentials and update handbook.
Before major release	Verify GitHub plan features, environment reviewers, branch rules and ruleset behavior.